

ASIGNACIÓN No. 5: INTEGRACIÓN DE IA MEDIANTE LLM Y MODEL CONTEXT PROTOCOL (MCP)

Plataforma e-Commerce Alelil Oficial | Broward International University (BIU)

Estudiante: Alberto Alfonso López Pereira	Curso: CSE6042_ Next-Gen Web Development
Profesor: Dr. Cristian Gabriel Zambrano Vega	Fecha de Entrega: Septiembre 2026
Stack: React.js, Node.js (Express), Gemini 1.5, Supabase, Azure	Servidor MCP: FastMCP / SDK Node.js (4 Herramientas)

1. Arquitectura General de la Solución (LLM + MCP + Azure)

La solución ha sido construida bajo una **arquitectura desacoplada de cuatro capas**, homologando el ecosistema completo en **Node.js (Express.js)** para permitir su despliegue óptimo en **Microsoft Azure (Azure App Services)** en alineación exacta con la propuesta de la Asignación #2.

Capa Lógica	Tecnología Seleccionada	Rol y Responsabilidad
1. Presentación (Frontend)	React.js SPA (Vite + CoreUI)	Interfaz web adaptativa e integración del widget flotante de Asistente IA (AlelilAIAssistant.jsx).
2. Orquestación & MCP Server	Node.js (Express.js Server)	Servidor MCP (server_mcp.js) desplegado en Azure App Service . Expone 4 herramientas REST y endpoint /api/chat.
3. Modelo Cognitivo (LLM)	Google Gemini (Gemini 1.5 Flash)	Modelo con soporte nativo de <i>Function Calling</i> para seleccionar e invocar herramientas del servidor MCP en tiempo real.
4. Persistencia & Seguridad	Supabase (PostgreSQL + Auth)	Base de datos relacional y autenticación con sistema RBAC (roles_usuario y función es_admin()).

2. Descripción del Modelo de Inteligencia Artificial Utilizado

Se seleccionó **Google Gemini (Gemini 3.6 Flash)** por ser el modelo multimodal insignia de última generación optimizado para baja latencia y ejecución eficiente de *Function Calling*. En lugar de generar respuestas especulativas (alucinaciones), Gemini analiza el prompt del usuario, identifica qué herramienta MCP requiere invocar (ej: buscar_productos) y extrae parámetros estructurados. Tras recibir los datos en JSON desde Supabase, redacta la respuesta conversacional final.

3. Explicación del Servidor MCP y Herramientas Mínimas Requeridas

El Servidor MCP (Model Context Protocol) se desarrolló en **Node.js** utilizando el estándar de declaración de herramientas que abstrae la capa de datos. Se implementaron exitosamente las **4 herramientas funcionales requeridas**:

Herramienta MCP	Firma / Parámetros	Operación en Supabase
-----------------	--------------------	-----------------------

consultar_productos_disponibles	{ limite: number }	Filtra y devuelve productos en catálogo con stock > 0.
buscar_productos	{ query: string, categoria: string }	Búsqueda relacional con filtros ilike en nombre o categoría.
consultar_precio_inventario	{ nombre_o_id: string }	Consulta puntual de precio exacto, estado de stock y disponibilidad.
recomendar_productos	{ presupuesto_max, categoria }	Algoritmo de recomendaciones ordenado por precio lte(presupuesto).

4. Medidas de Seguridad y Control de Acceso por Roles (RBAC)

Se resolvió de forma profesional el problema del correo hardcodedo (admin@tienda.com) reemplazándolo por un

****Control de Acceso Basado en Roles (RBAC)**:**

- **Tabla SQL roles_usuario:** Relaciona user_id (UUID) con el rol asignado (admin, cliente, vendedor).
- **Función de Seguridad SQL es_admin(uid):** Valida permisos dinámicamente en las políticas RLS.
- **Sanitización de Prompts:** El servidor MCP limpia la entrada del usuario eliminando comandos SQL o patrones de *Prompt Injection*.
- **Herramientas de Solo Lectura:** Las herramientas MCP ejecutan únicamente consultas SELECT, protegiendo la base de datos.

5. Fragmentos de Código Relevantes y Explicación Técnica

A. Definición del Esquema RBAC y Función de Seguridad en Supabase (SQL)

Explicación: Crea la tabla de roles y la función es_admin() para independizar los permisos del correo electrónico.

```
-- 1. Tabla de roles de usuario (RBAC)
CREATE TABLE IF NOT EXISTS public.roles_usuario (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID NOT NULL UNIQUE REFERENCES auth.users(id) ON DELETE CASCADE,
  rol TEXT NOT NULL CHECK (rol IN ('admin', 'cliente', 'vendedor'))
);

-- 2. Función helper de seguridad RLS
CREATE OR REPLACE FUNCTION public.es_admin(uid UUID) RETURNS BOOLEAN AS $$
BEGIN
  RETURN EXISTS (SELECT 1 FROM public.roles_usuario WHERE user_id = uid AND rol = 'admin');
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

B. Servidor MCP en Node.js - Registro y Ejecución de Herramientas (server_mcp.js)

Explicación: Conecta las herramientas MCP directamente con Supabase PostgreSQL y ejecuta Function Calling.

```
// Definición e implementación ejecutable de Herramientas MCP
async function ejecutarHerramientaMCP(nombreHerramienta, args = {}) {
  switch (nombreHerramienta) {
    case 'consultar_productos_disponibles':
      const { data } = await supabase.from('productos').select('*').gt('stock', 0).limit(args.limite || 10);
      return { status: 'ok', productos: data };

    case 'recomendar_productos':
      const query = supabase.from('productos').select('*').gt('stock', 0).lte('precio', args.presupuesto_max || 100);
      const { data: recs } = await query.order('precio', { ascending: false });
      return { status: 'ok', recomendaciones: recs };
  }
}
```

C. Configuración del LLM con Google Gemini 1.5 Flash y Function Calling

Explicación: Envía el prompt a Gemini incluyendo la declaración de herramientas MCP para invocar código del backend.

```
const aiClient = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

const response = await aiClient.models.generateContent({
  model: 'gemini-1.5-flash',
  contents: promptUsuario,
  config: {
    systemInstruction: 'Eres el Asistente IA de Alelil Oficial. Usa las herramientas MCP para responder.',
    tools: [{ functionDeclarations: mcpToolsDeclarations }]
  }
});
```

D. Componente de Interfaz del Asistente en React (AlelilAIAssistant.jsx)

Explicación: Renderiza el chat flotante, sugiere preguntas y permite agregar productos al carrito en 1-clic.

```
const enviarMensaje = async (prompt) => {
  const response = await fetch('http://localhost:8000/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: prompt })
  });
  const data = await response.json();
  setHistorial(prev => [...prev, { texto: data.reply, productos: data.productos_mcp }]);
};
```

6. Evidencias de Consultas Realizadas por Usuarios (Pruebas Reales)

Se ejecutaron y validaron las 4 consultas clave sobre la base de datos de Supabase:

Consulta del Usuario (Prompt)	Herramienta MCP Invocada	Respuesta Estructurada Generada
"¿Qué productos están disponibles?"	consultar_productos_disponibles	Devuelve catálogo en stock (Laptop ProCore 15 \$829, Monitor Vision 24 \$149, etc.).
"Busca productos en categoría Monitores"	buscar_productos	Filtra 2 monitores activos en Supabase (Vision 24 Full HD y UltraView 27 QHD).
"¿Cuál es el precio y stock de la Laptop ProCore 15?"	consultar_precio_inventario	Confirmó precio exacto de \$829 USD y 6 unidades disponibles en inventario.
"Recomiéndame productos por menos de \$300"	recomendar_productos	Recomendó Monitor UltraView 27 (\$269 USD) y Monitor Vision 24 (\$149 USD).

7. Repositorio GitHub e Instrucciones de Despliegue en Microsoft Azure

URL del Repositorio Público GitHub:

<https://github.com/oruebae/ecomerceCSE642>

Instrucciones de Despliegue en Microsoft Azure App Service:

- Configuración del Repositorio:** El proyecto incluye los archivos de despliegue de Azure web.config, process.json y la acción CI/CD de GitHub Actions en .github/workflows/azure-deploy.yml.
- Variables de Entorno en Azure:** Configurar en Azure App Service Application Settings:
 - GEMINI_API_KEY: Clave API de Google AI Studio.
 - VITE_SUPABASE_URL: URL del proyecto Supabase.
 - VITE_SUPABASE_ANON_KEY: Clave pública de Supabase.
- Ejecución Local de Pruebas:**
 - Backend & MCP Server: npm run start:backend (Puerto 8000)
 - Frontend React: npm run start (Puerto 3000 / Vite)